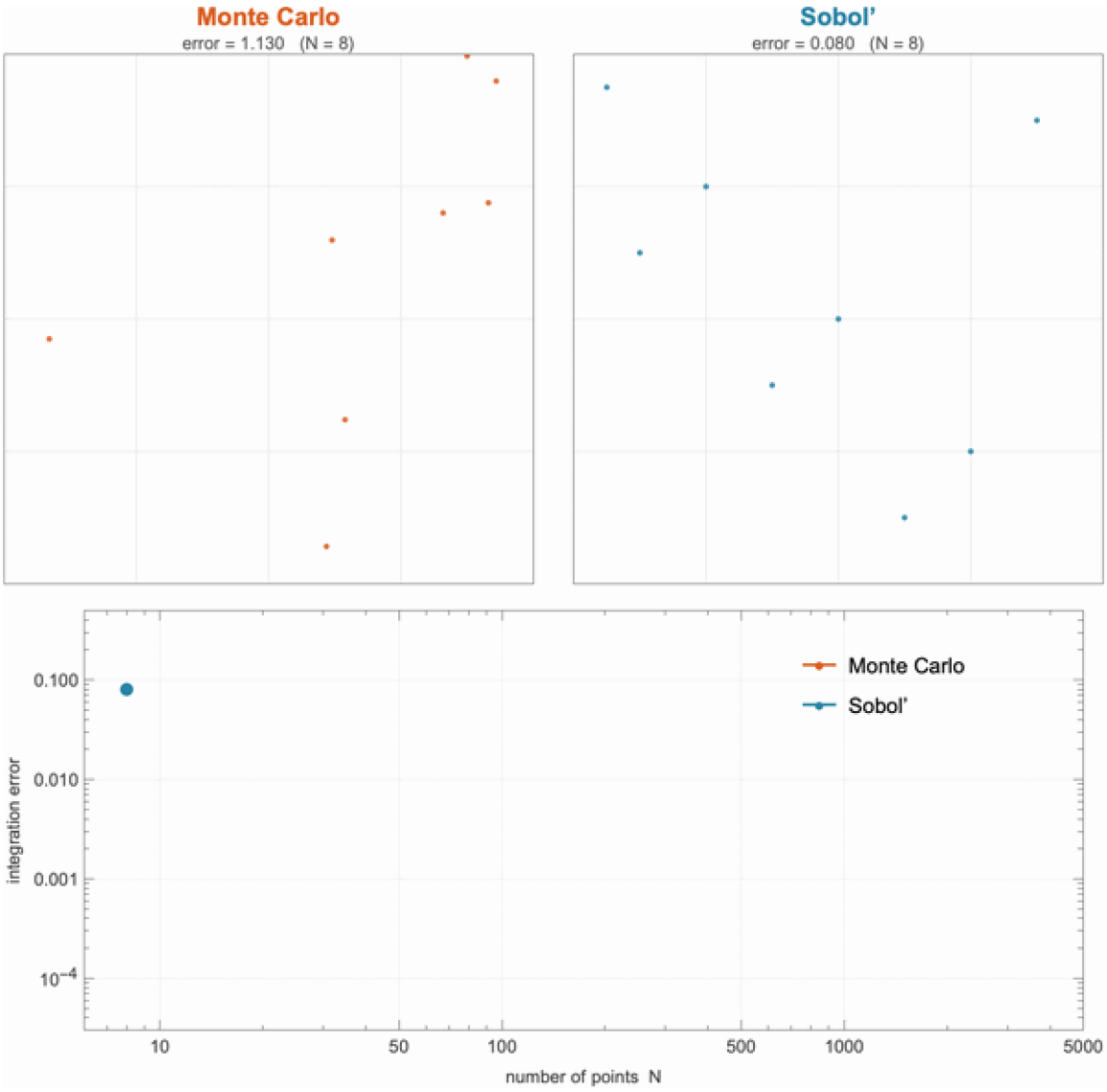

The whole post in one animation. Left: **Monte Carlo** points scattered at random clump and leave gaps. Right: **Sobol'** low-discrepancy points fill the same square evenly. Below: the error of a sample integral as the points accumulate — Monte Carlo crawls down the $1/\sqrt{N}$ line while Sobol' plunges along $1/N$. Generated near the end of this post with one call to `heroAnimation[]`.

Quasi-Monte Carlo for a real Bayesian estimation

Sobol' and Halton sequences against plain Monte Carlo on a six-parameter pharmacokinetic fit — and the textbook convergence gap, measured on real data

Almost every number a Bayesian reports — a posterior mean, a credible interval, a predicted drug concentration — is secretly an **integral** over the space of unknown parameters. This post is about computing those integrals an order of magnitude more cheaply.

The default tool is **Monte Carlo**: average the integrand over random samples. It is universal and assumption-free, but its error shrinks only as $N^{-1/2}$ — one more correct digit costs a hundred times more samples. **Quasi-Monte Carlo (QMC)** keeps the same averaging but replaces the random sample with a deterministic *low-discrepancy sequence* (Sobol', Halton) engineered to fill space evenly. For smooth integrands the error then falls almost as fast as $O\left(N^{-1}\,(\log N)^{d}\right)$ — nearly a full power of N better.

We test that promise on a genuine inference problem: a six-parameter pharmacokinetic model fit to the classic **Theophylline** data. We build Halton and Sobol' from scratch, set up the posterior, contrast it with classical curve-fitting, and measure the convergence. The headline, on real data: Monte-Carlo slopes near −0.5 against Sobol' slopes near −0.95, and roughly a 4×–18× saving in expensive model evaluations.

How to read this: every section opens with the plain idea, then the equations, the runnable Wolfram Language, and the measured numbers. All figures are pre-rendered — the post reads straight through — and every code cell runs if you load the attached package.

Setup

Skip this if you only want to read. To run the code, put the attached package `qmc_bayes_helpers.wl` next to the notebook and evaluate the three cells below once. They define the generators, the pharmacokinetic model, the data, the priors, and the posterior that every later section reuses.

```
SetDirectory[NotebookDirectory[]];
Get["qmc_bayes_helpers.wl"];
```

```
(* Lognormal-prior medians and log-scale SDs for {ka,CL,V1,Q,V2,sigma} *)
priorMedians = {1.5, 3.0, 35.0, 2.0, 25.0, 1.0};
priorLogSD   = {0.6, 0.5, 0.5, 0.7, 0.7, 0.5};

(* Real data: Theophylline Subject 1 (single 320 mg oral dose) *)
theoph = LoadTheoph[];
subj1  = SelectSubject[theoph, 1];
```

```
(* The Bayesian problem and its Laplace-Gaussian posterior *)
pkProblem = BayesProblem[subj1["dose"], subj1["t"], subj1["c"],
   priorMedians, priorLogSD];
pkLaplace = LaplacePosterior[pkProblem];
```

1. The square-root wall

Want the average of something over a space too big to enumerate? Sample it. Throw N random points, average the values. This Monte-Carlo estimate works in any dimension with no smoothness assumptions — which is why it is everywhere. The price is the error,

$$\epsilon_{\mathrm{MC}} \sim \frac{\sigma(f)}{\sqrt{N}} = O\left(N^{-1/2}\right)$$

Halving the error means quadrupling the work; one more decimal digit costs a hundredfold. When each sample is an expensive model solve, that square-root wall is the entire problem. Its cause is visible: random points clump and leave gaps, wasting information. What if we placed them on purpose?

```
mc  = LDPoints["MonteCarlo", 256, 2];
hal = LDPoints["Halton", 256, 2];
sob = LDPoints["Sobol", 256, 2];
GraphicsRow[ListPlot[#, AspectRatio -> 1, Frame -> True,
   PlotRange -> {{0, 1}, {0, 1}}, ImageSize -> 230] & /@ {mc, hal, sob}]
```

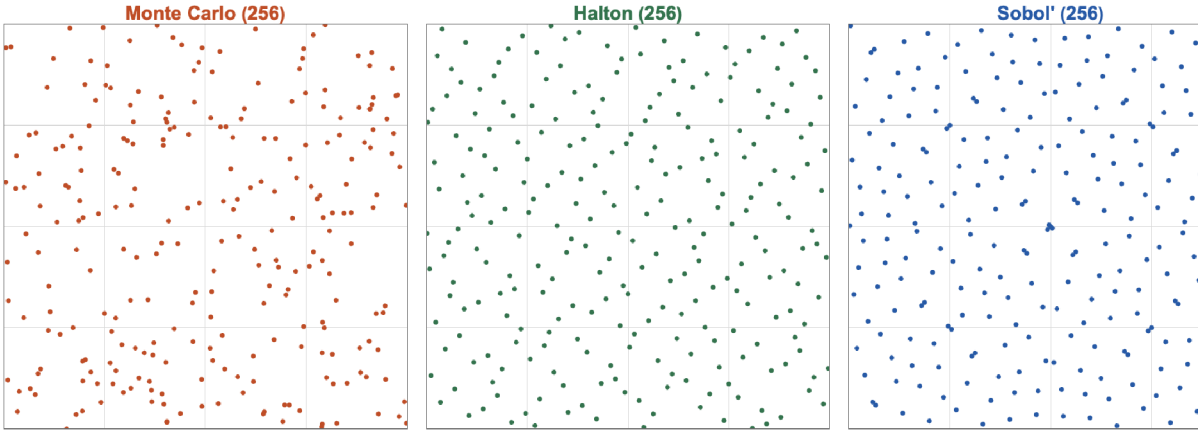

Figure 1. 256 points in the unit square. Monte Carlo (left) clumps and leaves holes; Halton (centre) and Sobol' (right) lay down an even, gap-free pattern.

2. Low-discrepancy sequences

A **low-discrepancy sequence** fills the cube as evenly as possible at every scale. **Halton** uses the radical inverse of the index in a different prime base per coordinate; **Sobol'** is a base-2 digital net built from "direction numbers" (we use the Joe–Kuo tables). Both are a few lines, both are in the attached package. Their L_2 **star discrepancy** — the worst box-vs-volume mismatch, by Warnock's exact formula — decays like $N^{-1/2}$ for random points but nearly N^{-1} for these sequences:

```
Nvals = 2^Range[4, 12];
dMC  = Mean[Table[StarDiscrepancyL2[
     RandomizedLDPoints["MonteCarlo", #, 2, 100 r + #]], {r, 20}]] & /@ Nvals;
dHal = StarDiscrepancyL2[LDPoints["Halton", #, 2]] & /@ Nvals;
dSob = StarDiscrepancyL2[LDPoints["Sobol", #, 2]] & /@ Nvals;
ListLogLogPlot[{Transpose[{Nvals, dMC}], Transpose[{Nvals, dHal}],
   Transpose[{Nvals, dSob}]}, Joined -> True, Mesh -> All]
```

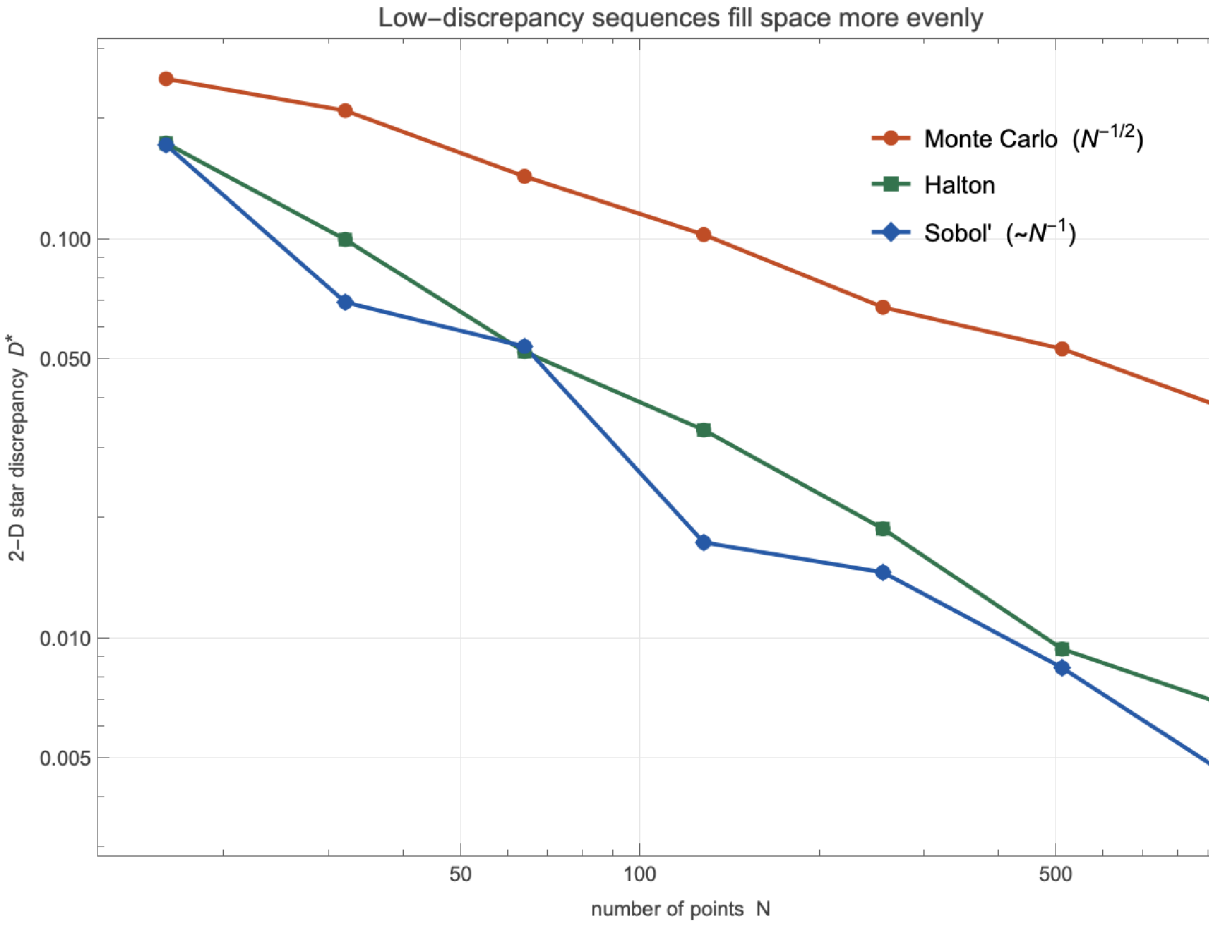

Figure 2. L_2 star discrepancy versus N (log-log). Monte Carlo rides the N^(-1/2) line; Halton and Sobol' track N^(-1).

The theorem behind it. For an integrand f of bounded variation $V(f)$ (Hardy–Krause), the **Koksma–Hlawka inequality** splits the QMC error into the function's variation times the points' discrepancy:

$$\left| \frac{1}{N} \sum_{n=1}^{N} f(x_n) - \int f(x)\, dx \right| \le V(f)\, D_N^*$$

With $D_N^* = O\left(N^{-1} (\log N)^d\right)$, QMC inherits that rate — *provided* the integrand is smooth enough to have finite variation. That proviso drives every modelling choice below.

3. The model: a drug moving through the body

What happens when you take a pill. The drug dissolves in the gut and is absorbed into the bloodstream; the blood carries it to the tissues, where some of it parks reversibly; meanwhile the liver and kidneys clear it away. Plot the blood concentration against time and you get a familiar shape: a fast rise to a peak, then a slow decline. **Pharmacokinetics** is the quantitative description of that

curve, and it is the backbone of every dosing decision — how much, how often, and how confident we are.

We use the canonical teaching data set, **Theophylline** (an anti-asthma drug, from Boeckmann, Sheiner & Beal): 12 subjects, each a single oral dose, serum concentration sampled 11 times over ~25 h. Theophylline has a narrow therapeutic window (about 10–20 mg/L; toxicity climbs above it), so getting the curve — and its uncertainty — right genuinely matters.

3.1 The two-compartment system

The standard structural model splits the body into two well-mixed *compartments*: a **central** one (blood and richly-perfused organs, volume V_1) where we actually measure concentration, and a **peripheral** one (slower tissues, volume V_2). The swallowed dose sits in a gut depot A_g and is absorbed first-order into the central compartment; drug shuttles between central and peripheral, and is cleared from the central compartment. Writing the amounts in each compartment, the dynamics are a linear system of ODEs:

$$\frac{dA_g}{dt} = -k_a A_g$$

$$\frac{dA_1}{dt} = k_a A_g - \frac{\mathrm{CL}}{V_1} A_1 - \frac{Q}{V_1} A_1 + \frac{Q}{V_2} A_2$$

$$\frac{dA_2}{dt} = \frac{Q}{V_1} A_1 - \frac{Q}{V_2} A_2$$

with the oral dose as initial condition $A_g(0) = D$, $A_1(0) = A_2(0) = 0$, and the measured concentration $C(t) = A_1 / V_1$.

3.2 The six parameters, one by one

These are the unknowns we infer — each a physical quantity a clinician can interpret:

- ka — **absorption rate** (1/h). How fast the drug leaves the gut for the blood; it sets the steepness of the rising limb and the time-to-peak.
- CL — **clearance** (L/h). Volume of blood completely cleared of drug per hour. It alone fixes the total exposure AUC = D / CL — the single most important dosing parameter.
- V1 — **central volume** (L). Relates dose to the initial concentration; small V_1 means a high peak.
- Q — **inter-compartmental clearance** (L/h). How fast drug exchanges with the tissues; it shapes the bend between the fast and slow phases.
- V2 — **peripheral volume** (L). How much drug the tissues hold; it stretches the slow elimination tail.
- σ — **residual scatter** (mg/L). The combined assay-plus-model noise on each measurement — itself an unknown to be estimated.

From these follow the quantities dosing actually turns on: half-life $t_{1/2} \propto V_1 / \mathrm{CL}$, exposure D / CL, peak concentration, and the steady state reached under repeated dosing.

3.3 A closed form — no solver needed

Because the system is linear, it has an exact solution: a sum of three exponentials in the macro-rate constants α, β (the eigenvalues of the compartment matrix) and k_a. This makes each likelihood evaluation a handful of Exps rather than an ODE solve:

$$C(t) = \frac{k_a D}{V_1}\left(A_\alpha e^{-\alpha t} + A_\beta e^{-\beta t} + A_k e^{-k_a t}\right)$$

```
(* The closed-form concentration; ka, CL, V1, Q, V2, dose. *)
Plot[PKConcentration[t, {1.5, 2.8, 30., 2.0, 20.}, 320.], {t, 0, 25},
   Frame -> True, FrameLabel -> {"time (h)", "conc (mg/L)"},
   PlotStyle -> RGBColor[0.153, 0.51, 0.64]]
```

Why bother with six parameters? A simpler one-compartment model has three. The second compartment is what lets the curve bend — a fast distribution phase then a slow elimination phase — which real drugs do. The cost is that the two tissue parameters (Q, V2) are only weakly pinned down by a single subject's sparse samples. Holding honest uncertainty about them, rather than a false-precision point estimate, is exactly where the Bayesian treatment earns its keep.

4. The Bayesian model, and why it is all integrals

Each measurement is the model prediction plus Gaussian scatter, $c_i = C(t_i) + \epsilon_i$ with $\epsilon_i \sim N(0, \sigma^2)$. All six parameters are positive, so each gets a **lognormal** prior (a normal prior on its logarithm). Bayes' rule gives the posterior:

$$p(\theta \mid c) \propto \prod_{i=1}^{n} N\left(c_i \mid C(t_i), \sigma^2\right) \pi(\theta)$$

Now the key point. Everything we want from this posterior is an **expectation** — an integral over the six-dimensional parameter space:

$$E_{\text{post}}[g(\theta)] = \int g(\theta)\, p(\theta \mid c)\, d\theta$$

Posterior mean clearance? Take $g = \mathrm{CL}$. Its uncertainty? $g = \mathrm{CL}^2$. Probability the half-life exceeds a threshold? g an indicator. The predicted concentration at the next clinic visit? $g = C(t_*)$. Six-dimensional integrals of a smooth density, with no closed form: the exact setting where the choice of integration rule — Monte-Carlo or quasi-Monte-Carlo — decides the cost.

5. Classical curve-fitting versus the Bayesian posterior

The classical reflex. Hand this data to a statistician and the first move is **nonlinear least squares** — in Wolfram Language, NonlinearModelFit — which slides the six parameters until the squared residuals are smallest. It is fast and gives a tidy best-fit curve:

```
model = PKConcentration[t, {ka, CL, V1, Q, V2}, subj1["dose"]];
nlm = NonlinearModelFit[Transpose[{subj1["t"], subj1["c"]}], model,
   {{ka, 1.5}, {CL, 2.8}, {V1, 30}, {Q, 2.0}, {V2, 20}}, t];
{nlm["BestFitParameters"], nlm["ParameterErrors"]}
```

And the trouble it hides. The point estimate is fine, but ask for its error bars and the asymptotic standard errors come back enormous — of order 10^6–10^7 on CL, V_1, V_2, with V_2 driven to essen-

tially zero. The reason is real: a two-compartment model is **not identifiable** from one subject's eleven points, so the least-squares surface is nearly flat in some directions and the linearized covariance blows up. Classical fitting gives you a curve but no trustworthy uncertainty.

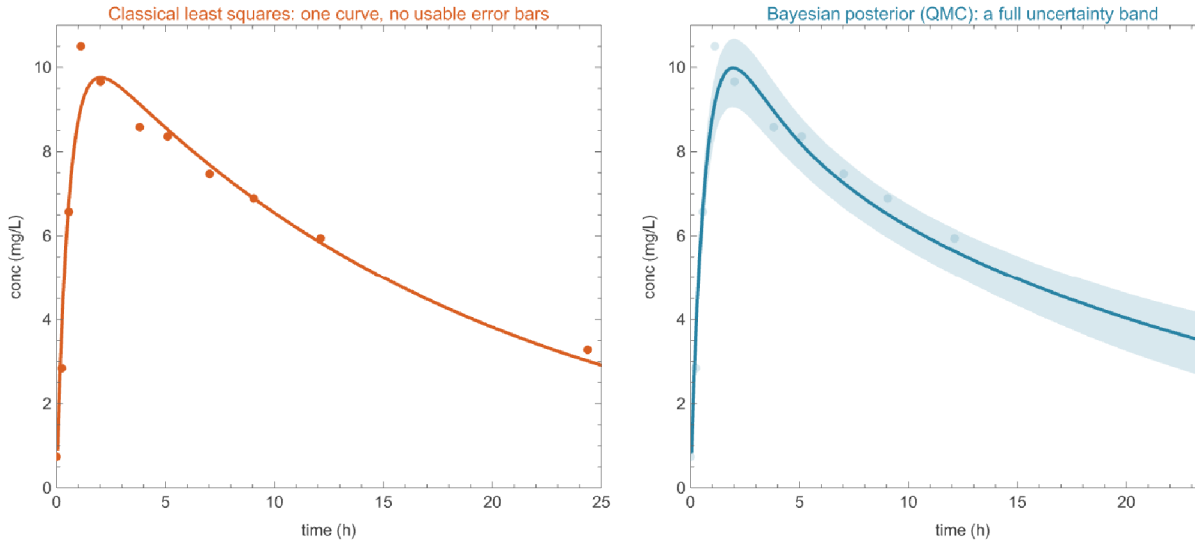

Figure 3. Same data, two philosophies. ***Left:*** *least squares returns a single best-fit curve — and, because the model is unidentifiable here, no usable error band.* ***Right:*** *the Bayesian posterior (sampled by QMC) delivers a full predictive band; the lognormal priors gently regularize the directions the data cannot constrain.*

The Bayesian posterior fixes this honestly: the priors keep the weakly-identified directions finite, and the answer is a full joint distribution — every parameter, every prediction, with calibrated uncertainty. The catch is the one from §4: you must integrate. The options:

- **Grid quadrature** — exact-ish in 1–2 dimensions, hopeless in six (10 points / dim is 10^6 evaluations, and the accuracy is still poor).
- **Markov-chain Monte Carlo (MCMC)** — the standard Bayesian workhorse; it samples the posterior directly but inherits the $N^{-1/2}$ Monte-Carlo rate (worse, after autocorrelation) and needs tuning and convergence checks.
- **Laplace approximation** — fit a Gaussian at the posterior mode; cheap, and the basis we build on here.
- **Quasi-Monte Carlo** — deterministic low-discrepancy points, near N^{-1} for smooth integrands. For moderate-dimensional, smooth posteriors it is the most evaluations-efficient of the lot — and it is a drop-in replacement for the random draws.

QMC does not compete with MCMC so much as complement it: where you can reparameterize the posterior to be smooth and roughly Gaussian (Bayesian asymptotics says you usually can), QMC integration buys back most of the lost power. That is the recipe we follow next.

6. The fit

From posterior to a unit-cube integral. We build the **Laplace–Gaussian** approximation — posterior mode plus curvature in log-parameter space, a multivariate normal $N(\mu, \Sigma)$ on $\varphi = \log\theta$ — and map the unit cube onto it with the inverse-CDF transform $\varphi(u) = \mu + L\,\Phi^{-1}(u)$, $L\,L^{T} = \Sigma$. A posterior expectation becomes an integral over $[0, 1]^{6}$ of a smooth, weight-free integrand — ready for QMC.

```
(* Draw from the posterior with a Sobol' set; form the predictive band *)
draws  = PosteriorDraws[LDPoints["Sobol", 4096, 6], pkLaplace];
tGrid  = Subdivide[0., 25., 200];
curves = ConcentrationCurves[draws, tGrid, subj1["dose"]];
band   = Transpose[curves];
{Quantile[#, 0.05] & /@ band, Quantile[#, 0.95] & /@ band};
```

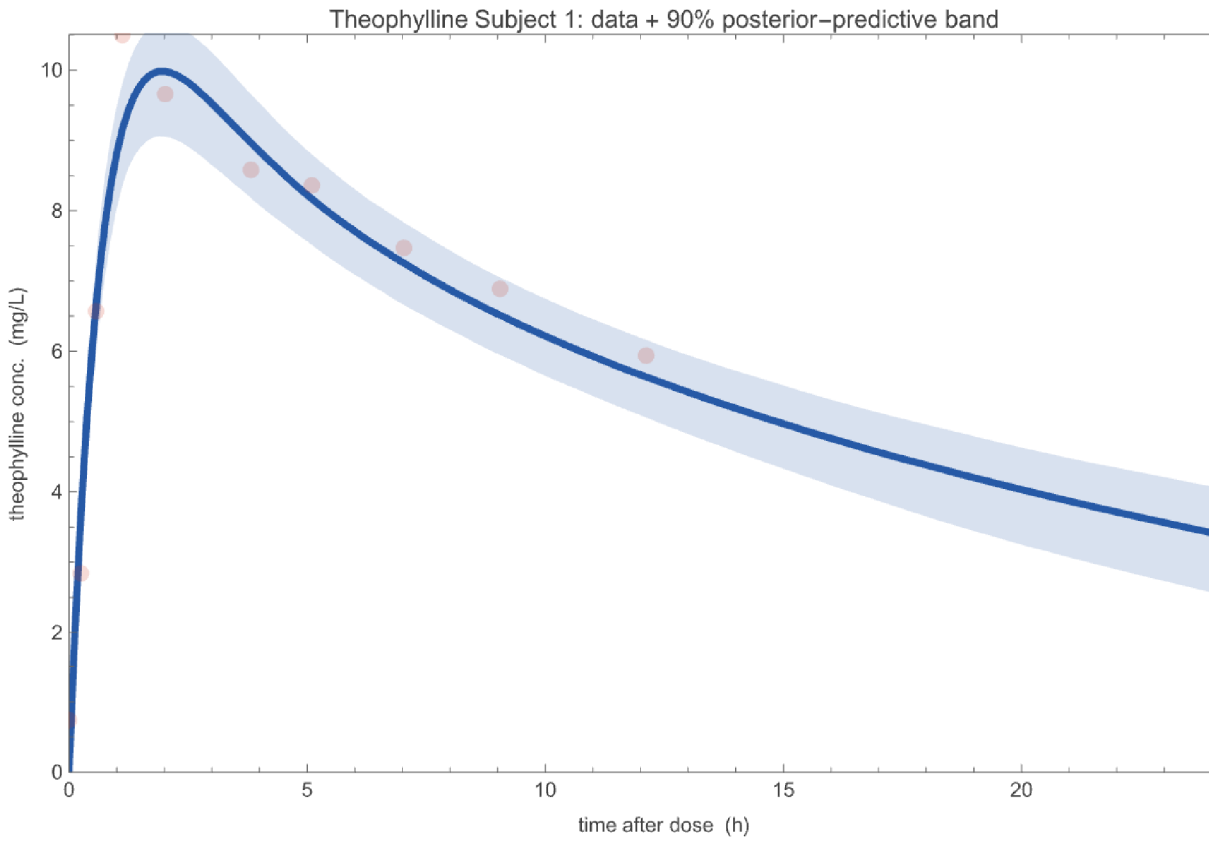

Figure 4. Theophylline Subject 1: the 11 measurements with the 90% posterior-predictive band. The fit captures the absorption peak near 2 h and the slow elimination tail.

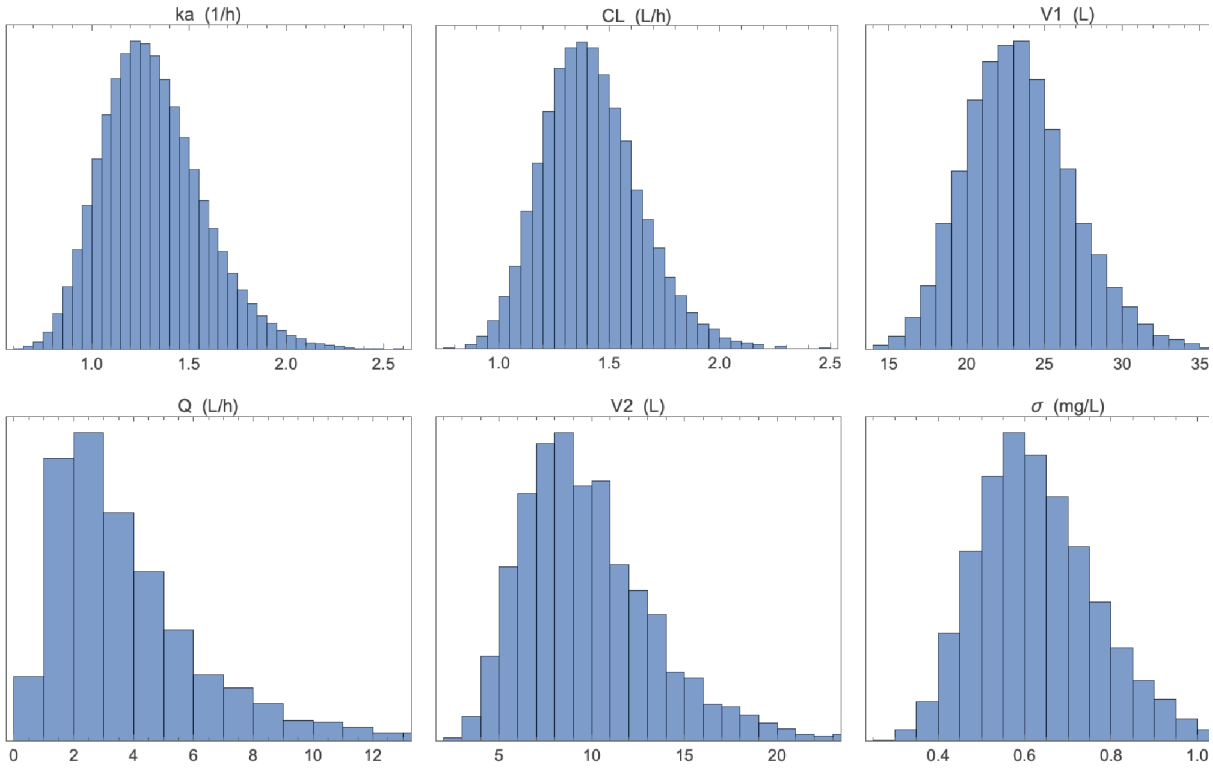

Figure 5. Posterior marginals. Clearance CL, central volume V_1, and absorption k_a are tightly determined; Q and V_2 stay broad — honest uncertainty where the data are thin.

Is the Gaussian approximation good enough? We check against the full posterior via importance sampling (`FitProposal` + `ImportanceEstimate`): the well-identified parameters agree to ~10%. And the convergence demonstration does not depend on the approximation being perfect — only on the integrand we actually integrate being smooth, which it is.

7. The convergence law

Now the measurement. We estimate two posterior expectations and watch the error fall with the number of points, for Monte Carlo, Sobol', and Halton: the **mean clearance** $E_{\text{post}}[\text{CL}]$ (which has an analytic value under the Gaussian posterior, so we know the truth exactly) and the **predicted concentration at 6 h** (no closed form — each evaluation a model solve). Errors are averaged over independent randomizations (a digital shift for Sobol', a Cranley–Patterson rotation for Halton):

```
Nlist = 2^Range[7, 15];
refCL = LaplaceMeanExact[pkLaplace][[2]];          (* exact E[CL] *)
gCL   = #[[All, 2]] &;
rmseCL = Association[# -> ExpectationRMSE[#, Nlist, 200, pkLaplace, gCL, refCL] &
   {"MonteCarlo", "Sobol", "Halton"}];
ListLogLogPlot[Table[Transpose[{Nlist, rmseCL[m]}],
   {m, {"MonteCarlo", "Sobol", "Halton"}}], Joined -> True, Mesh -> All]
```

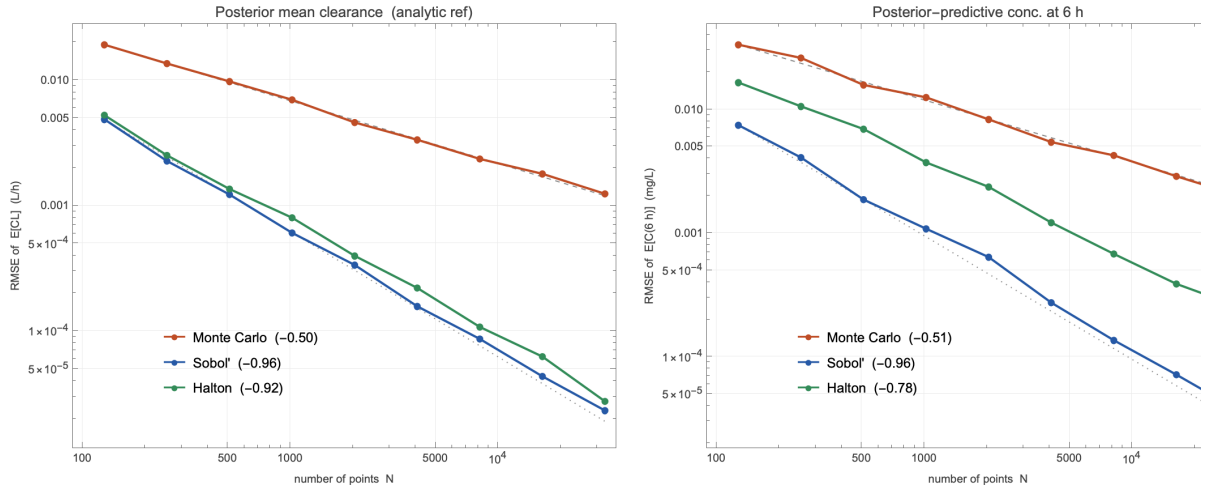

Figure 6. RMSE versus N (log-log). ***Left:*** *posterior mean clearance (analytic reference).* ***Right:*** *posterior-predictive concentration at 6 h. Monte Carlo follows N^(-1/2) (slope ≈ -0.50); Sobol' and Halton follow N^(-1) (slopes ≈ -0.95). At N = 32768 Sobol' is ~50× more accurate, and the gap keeps widening.*

That is the headline, on real data: the same number of model solves buys nearly twice the correct digits with Sobol' as with Monte Carlo.

8. Why dimension matters

The $(\log N)^d$ factor is not cosmetic: filling a cube evenly gets exponentially harder as the dimension grows, so the QMC advantage erodes. Fix the budget N and measure how many times more accurate Sobol' is than MC on a smooth test integral, across dimension:

```
cAmp  = 0.6;
fMean = Function[pts, Mean[Times @@@ (1 + cAmp Cos[2 Pi pts])]];   (* exact 1 *)
rmseAt[meth_, d_, n_] := Sqrt@Mean@Table[
   (fMean[RandomizedLDPoints[meth, n, d, 53 r + d]] - 1.)^2, {r, 120}];
gain[n_] := Table[rmseAt["MonteCarlo", d, n]/rmseAt["Sobol", d, n], {d, 2, 12}];
{gain[4096], gain[65536]}
```

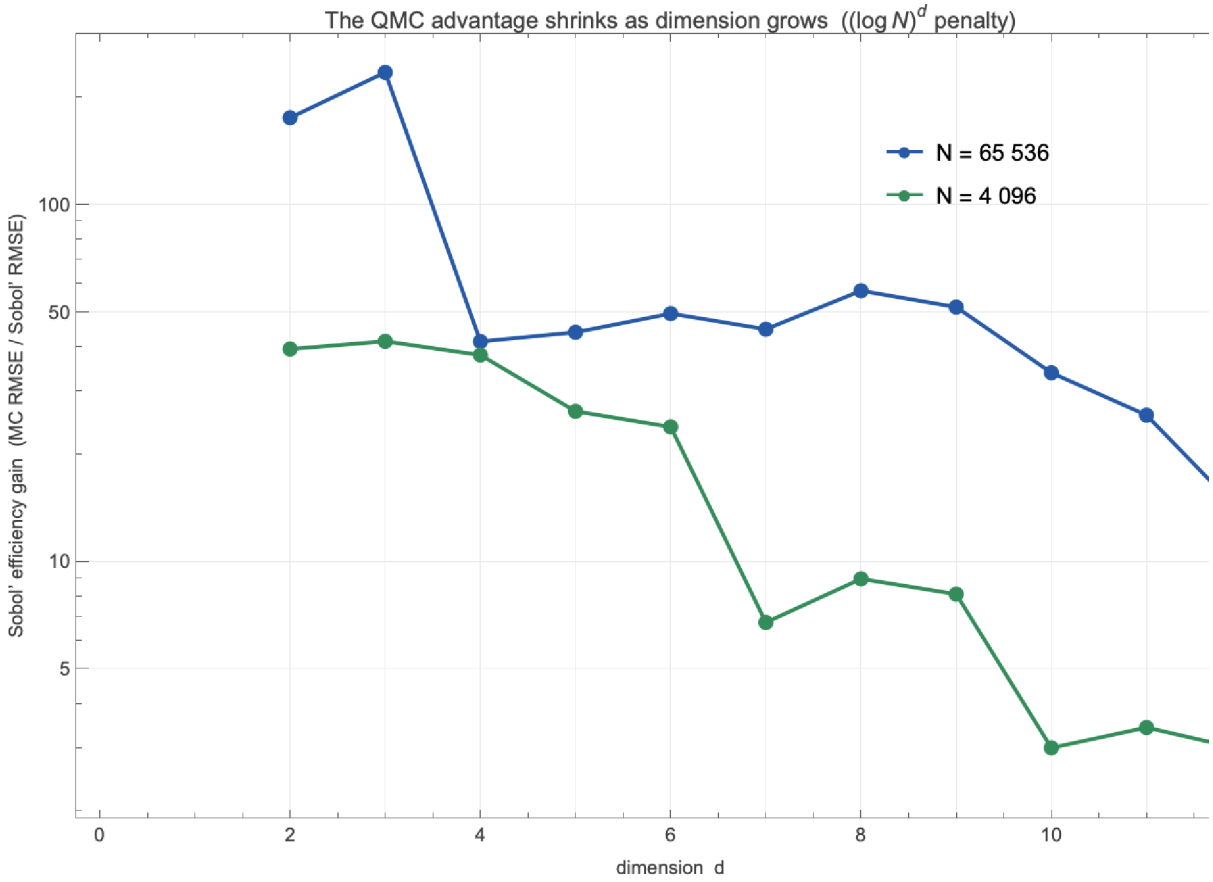

Figure 7. Sobol' efficiency gain (MC RMSE / Sobol' RMSE, fixed budget) versus dimension. Tens to hundreds of times more accurate in low dimension, decaying toward parity as the (log N)\^d penalty bites. The six-parameter PK problem sits where the gain is still large.

9. Does it recover the truth?

A fast wrong answer is worthless, so we generate data from known parameters, fit, and check the posterior brackets them:

```
thetaTrue = {1.20, 2.50, 30.0, 1.80, 22.0, 0.45};
{synProblem, synConc} = SyntheticProblem[thetaTrue, 320.0,
   {0.25, 0.5, 1, 1.5, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24, 30},
   priorMedians, priorLogSD, 4242];
synDraws = PosteriorDraws[LDPoints["Sobol", 8192, 6],
   LaplacePosterior[synProblem]];
Mean /@ Transpose[synDraws]
```

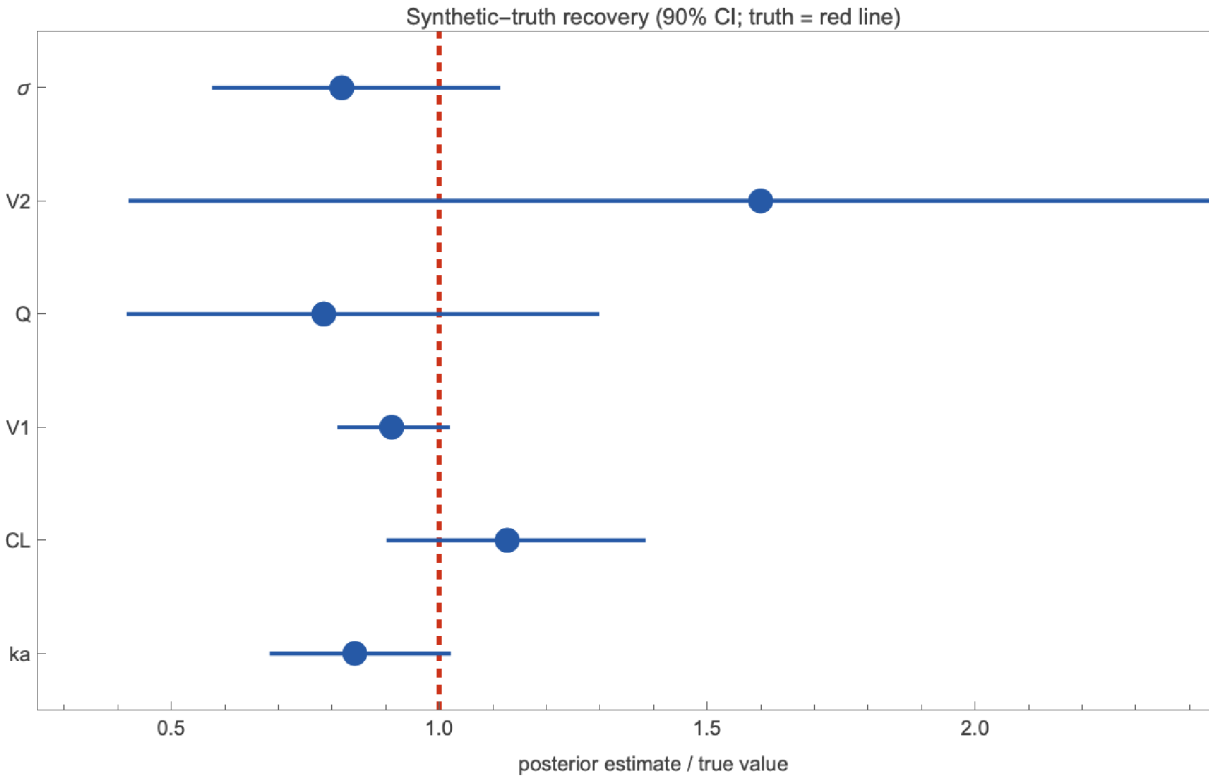

Figure 8. Posterior estimate / true value, with 90% credible intervals (red line = truth). The well-identified parameters (k_a, CL, V_1) land on the truth; Q and V_2 carry wide intervals that still cover it. Correct, and honest about what the data cannot say.

10. The practical pay-off

Slopes are abstract; model solves are money. Inverting the fitted error laws — $N \propto \text{tol}^{-2}$ for Monte Carlo, $N \propto \text{tol}^{-1}$ for QMC — gives the number of forward-model evaluations needed for a target accuracy, and the ratio is the speed-up:

```
fitPow[e_] := Module[{c = Fit[Transpose[{Log[Nlist], Log[e]}], {1, x}, x]},
   {Exp[c /. x -> 0], Coefficient[c, x]}];
nForTol[e_, tol_] := With[{cp = fitPow[e]}, (cp[[1]]/tol)^(1/(-cp[[2]]))];
nForTol[rmsePred["MonteCarlo"], 0.001 refPred] /
   nForTol[rmsePred["Sobol"], 0.001 refPred]
```

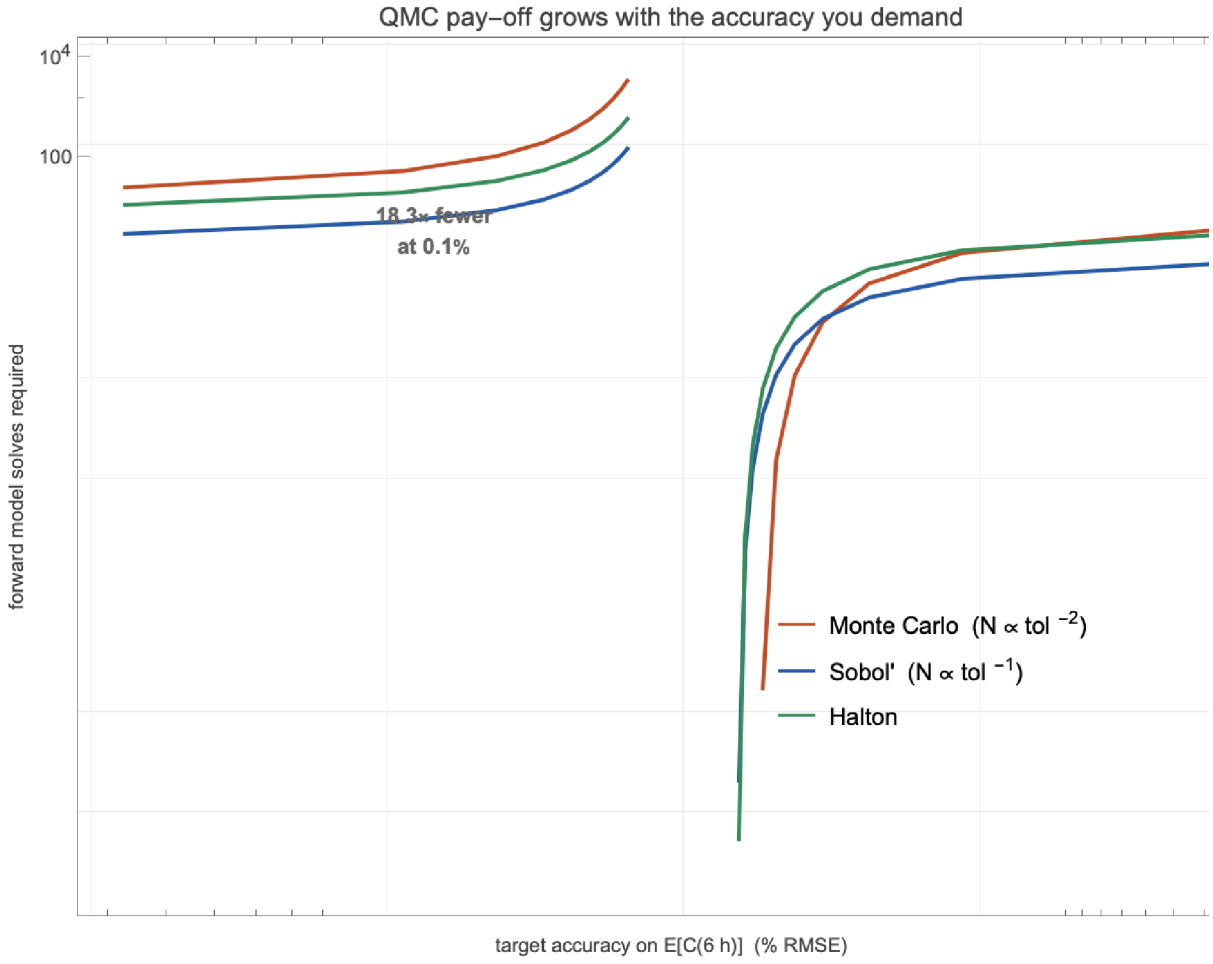

Figure 9. Forward-model solves to reach a target accuracy on the predicted concentration. Because MC needs O(tol^(-2)) and QMC O(tol^(-1)), the gap widens with the accuracy demanded: ~2× at 1%, ~4× at 0.5%, ~18× at 0.1%.

That is the value proposition in one figure. QMC changes neither the model nor the estimand — only *which points you evaluate at* — and that single swap buys back most of an order of magnitude in expensive model runs.

11. The cover animation

For completeness, here is the code behind the animation at the top — the same Monte-Carlo and Sobol' point sets accumulating, with the running integration error traced beneath. It is the whole post compressed into $1/N$ versus $1/\sqrt{N}$:

```
frameNs = Round[2^Range[3, 12]];
fInt[pt_] := Exp[Total[pt]];   exact = (E - 1.)^2;
mcAll  = BlockRandom[SeedRandom[7]; RandomReal[1, {4096, 2}]];
sobAll = LDPoints["Sobol", 4096, 2];
runErr[pts_] := Abs[Mean[fInt /@ pts] - exact];
```

```
(* each frame: MC cloud | Sobol cloud, with the error-vs-N race below *)
frames = Table[ Rasterize @ Column[{
     Row[{ListPlot[mcAll[[1 ;; n]]], ListPlot[sobAll[[1 ;; n]]]}],
     ListLogLogPlot[{ {#, runErr[mcAll[[1 ;; #]]]} & /@ Select[frameNs, # <= n &],
                      {#, runErr[sobAll[[1 ;; #]]]} & /@ Select[frameNs, # <= n &]
},
        Joined -> True] }], {n, frameNs}];
Export["hero.gif", AnimatedImage[frames, FrameRate -> 1.4]]
```

12. Conclusions

On a real six-parameter pharmacokinetic inference, deterministic low-discrepancy sampling beats plain Monte Carlo by close to a full power of N — measured slopes near −0.95 against −0.5, and an order-of-magnitude saving in model evaluations at useful accuracies. Against classical least squares, the Bayesian treatment adds honest, regularized uncertainty where an unidentifiable model would otherwise report nonsense error bars — and QMC is what makes computing that uncertainty cheap.

When QMC helps, and when it does not. It helps when the dimension is low-to-moderate or the effective dimension is small, the integrand is smooth, and evaluations are the bottleneck. It does not help for rough or discontinuous integrands, very high dimension, or heavy-tailed importance weights — which is why we integrate against a smooth Gaussianized posterior rather than a spiky ratio.

The portable recipe. (1) reparameterize to an unconstrained space and build a Gaussian approximation of the posterior; (2) map the unit cube through it with the inverse-CDF transform; (3) swap your pseudo-random draws for a randomized Sobol' or Halton set; (4) average over a few randomizations for an error bar. Two lines of change, most of an order of magnitude back.

13. References & reproducibility

Halton, J. H. (1960). *Numerische Mathematik* 2, 84–90. • **Sobol', I. M.** (1967). *USSR Comp. Math. Math. Phys.* 7, 86–112. • **Joe, S., & Kuo, F. Y.** (2008). *SIAM J. Sci. Comput.* 30, 2635–2654 (the direction numbers used here).

Niederreiter, H. (1992). *Random Number Generation and Quasi-Monte Carlo Methods*, SIAM. • **Caflisch, R. E.** (1998). *Acta Numerica* 7, 1–49 (effective dimension). • **Owen, A. B.** (1998). *J. Complexity* 14, 466–489 (randomized QMC). • **Gerber, M., & Chopin, N.** (2015). *JRSS B* 77, 509–579 (QMC in Bayesian computation).

Boeckmann, Sheiner & Beal (1994). *NONMEM Users Guide* (the Theophylline data). • **Gabrielsson & Weiner** (2016). *Pharmacokinetic and Pharmacodynamic Data Analysis* (the two-compartment model).

All code is pure Wolfram Language at **github.com/mthiel74/QuasiMonteCarloBayesianEstimation** (`https://github.com/mthiel74/QuasiMonteCarloBayesianEstimation`). The companion package `qmc_bayes_helpers.wl` is the single source of truth; the figure scripts and this notebook both load it. To rebuild:

```
wolframscript -file wolfram/fetch_theoph.wls   # real data -> data/
wolframscript -file wolfram/run_all.wls        # all figures -> docs/images/
wolframscript -file community/build_notebook.wls
```

Generated June 4, 2026.